

STABLE COIN ARBITRAGE

SIMON FRASER UNIVERSITY - INTELLIGENT SYSTEMS

KEVIN LITVIN, ARIO KATCHOOI, ALISH SINGH

PROF: HANG MA

DEC 14, 2025

Stablecoin Cross-Exchange Arbitrage

Table of Contents

1. Introduction

- 1.1..... Mathematical Formulation of Cross-Exchange Stablecoin Arbitrage
- 1.2A* Search Formulation
- 1.3Heuristic Functions
 - 1.3.1 h1 – Liquidity Heuristic
 - 1.3.2 h2 – Slippage Heuristic
 - 1.3.3 h3 – Parallel Execution Heuristic
 - 1.3.4 h4 – Chain Congestion and Exchange Reliability Heuristic

2. Implementation

- 2.1..... Market Data Layer
- 2.2Fee Model
- 2.3Transfer Latency Model
- 2.4.....Graph Construction
- 2.5.....A* Search with Liquidity/Slippage
- 2.6..... Weighted A* with Risk-Aware Heuristic
- 2.7..... Liquidity Heuristic Implementation
- 2.8.....Slippage Heuristic Implementation
- 2.9..... Parallel Multi-Start Wrapper
- 2.10... Chain Congestion and Exchange Reliability Heuristic

3. Methods

- 3.1..... Research Questions
- 3.2..... Algorithmic Behavior and Expected Suitability of Each Heuristic
- 3.3..... Classes of Problem Instances

4. Experimental Setup and Evaluation Framework

- 4.1..... Unit Tests and Correctness Validation
- 4.2..... Deterministic Heuristic Comparison
- 4.3..... Monte Carlo Robustness Experiments
- 4.4..... Interactive Real-Time Validation – Streamlit UI

5. Results

- 5.1..... Deterministic Heuristic Comparison
- 5.2..... Stochastic Evaluation with Monte Carlo Simulation

6. Conclusion

7. Bibliography

1 Introduction

Cross-exchange stablecoin arbitrage exploits price differences for the same asset across centralized exchanges. Although the idea is simple, buy on the cheaper exchange and sell on the more expensive one, practical execution is limited by fees, slippage, transfer delays, liquidity, exchange risk, and the short time window in which profits remain viable. Manually identifying profitable multi-exchange cycles is slow, error-prone, and not scalable.

This project develops an automated arbitrage route planner that finds full profitable trading cycles—trades *and* transfers—rather than simple pairwise price gaps. By incorporating real trading fees, withdrawal fees, transfer times, and live market data through CCXT, the system generates execution-ready arbitrage paths and reduces trader workload while improving decision reliability.

Our work extends prior research such as Crypto Advisor by Oantă & Coroiu (2023), which detects price discrepancies but does not compute full trade-and-transfer cycles or model execution risk. Unlike these earlier approaches, we formulate arbitrage search as a graph-planning problem and use A* with domain-specific heuristics. We integrate real-world constraints including slippage, liquidity depth, chain congestion, and exchange reliability.

Through controlled experiments and Monte Carlo simulations, we evaluate four custom heuristics—liquidity, slippage, parallel search, and chain-plus-exchange-risk—and analyze their impact on performance and profitability. Ultimately, our system provides complete, executable arbitrage routes, offering a more practical and reliable foundation for real-time cross-exchange trading.

1.1 Mathematical Formulation

We formulate the stablecoin cross-exchange arbitrage problem as a directed weighted graph $G = (V, E)$. Each node $v \in V$ represents a unique stablecoin–exchange pair, written as $v = (exchange, coin)$. Each directed edge $e \in E$ corresponds either to a trade executed on an exchange or a transfer of funds between exchanges. Every edge has an associated effective exchange rate that incorporates all relevant costs—including trading fees, withdrawal fees, spreads, and slippage—and we assign each edge a weight using the negative logarithm of this effective rate:

$$w(e) = -\log(effective_rate(e))$$

This transformation converts multiplicative returns into additive costs, enabling the use of shortest-path algorithms to evaluate profitability.

Given a path $P = (v_0, v_1, \dots, v_k)$ with initial capital C_0 the final capital after executing all trades and transfers along the path is

$$C_k = C_0 * \prod_{e \in P} r(e)$$

where $r(e)$ denotes the effective rate on edge e . A path is profitable when the net gain exceeds a user-specified minimum profit threshold:

$$C_k - C_0 \geq \text{min_profit}$$

Expressing this condition in additive log space gives an equivalent constraint:

$$\sum_{e \in P} -\log(r(e)) \leq -\log\left(1 + \frac{\text{min_profit}}{C_0}\right)$$

Thus, the arbitrage search problem becomes the task of identifying a cycle that begins and ends at a given start node v_0 and yields a multiplicative return greater than one. More formally, we seek:

$$\text{Find } P: v_0 \rightarrow v_0 \text{ s.t. } \prod_{e \in P} r(e) > 1$$

This constitutes a minimum-cost cycle search under realistic execution constraints.

A* Search Formulation

To solve the arbitrage problem efficiently, we apply the A* search algorithm. For each node n , we maintain a cost-to-reach value $g(n)$, computed as the sum of the additive edge costs encountered so far:

$$g(n) = \sum_{e \in \text{path}} -\log(r(e))$$

We also define a heuristic function $h(n)$ that estimates the remaining cost or risk associated with completing a profitable cycle. The A* evaluation function is then:

$$f(n) = g(n) + h(n)$$

The algorithm terminates when it returns to the starting node v_0 with final capital exceeding the initial capital, i.e. $C_n > C_0$. Through this formulation, A^* is able to balance actual accrued costs with heuristic predictions of remaining feasibility and risk.

Heuristic Functions (h1-h4)

H1 - liquidity

We designed four domain-specific heuristics to guide the search toward reliable, low-risk, and realistically executable arbitrage routes.

The h1 liquidity heuristic models the risk that low-volume markets cannot support the required trade size without excessive slippage. Let

$$vol_per_sec = \frac{24h_quote_volume}{24 * 3600}, \quad liquidity_ratio = \frac{vol_per_sec * T_{remain}}{order_size_usd}$$

We map this value to a normalized liquidity score:

$$score_{liq} = clamp(\frac{\log_{10}(liquidity_ratio)+1}{2}), 0, 1),$$

and define the heuristic cost as:

$$h1(n) = \lambda_{liq} (1 - score_{liq}(n)).$$

h2 - slippage heuristic

The h2 slippage heuristic accounts for expected execution price impact based on real order-book depth. Let P_{mid} be the mid-price and $VWAP(Q)$ the volume-weighted average price needed to fill order size Q . The slippage in basis points is:

$$slippage_{bps} = \frac{VWAP(Q) - p_{mid}}{p_{mid}} * 10^4$$

And the heuristic is:

$$h_2(n) = \lambda_{slip} * \max \{0, slippage_{bps} - \theta\}$$

Where θ is a user- defined slippage tolerance.

h3 - Parallel execution heuristic

The h3 parallel-execution heuristic models the timing of arbitrage execution rather than cost. It determines whether trade and transfer operations can overlap within the allowable arbitrage window. Since it tracks wall-clock execution rather than a scalar cost, it does not admit a closed-form expression but influences feasibility pruning.

h4 - Chain Congestion + Exchange Reliability heuristic

The h4 chain-congestion and exchange-reliability heuristic incorporates delays and risks associated with blockchain transfers and exchange stability. If t_{min} is the shortest outgoing transfer time and T_{remain} the remaining arbitrage window, then the chain reliability score is:

$$score_{chain} = 1 - \frac{t_{min}}{T_{remain}}$$

With heuristic cost

$$h_{chain}(n) = \lambda_{chain}(1 - score_{chain}(n))$$

If each exchange has a reliability score $s(n) \in [0, 1]$, we define:

$$h_{exchange}(n) = \lambda_{exchange}(1 - s(n))$$

The combined risk-aware heuristic is:

$$h_4(n) = h_{chain}(n) + h_{exchange}(n),$$

Which biases the search towards fast, reliable, low-risk execution paths.

2. Implementation

2.1 Implementation - data.py

The `data.py` module defines the live market data layer that feeds our arbitrage graph, using the CCXT library to connect to four centralized exchanges (Binance, Kraken, KuCoin, and Bybit) and query their public tickers in real time. It first declares the set of stablecoins we track (USDT, USDC, DAI, TUSD, FDUSD, BUSD, PYUSD, USDP, GUSD, FRAX), then specifies, for each coin–exchange pair, which spot market symbol to use in CCXT (for example, USDT/USD on Kraken or USDC/USDT on Binance). These mappings were constructed from each platform’s official market listings and documentation pages so that only actually listed pairs are queried. The helper `normalize_price_to_usd` converts each mid-price into a consistent “1 coin ~ X USD” value by applying simple rules: if the quote is USD, the price is already in dollars; if the quote is another stablecoin (USDT, USDC, FDUSD, etc.), it is treated as approximately 1 USD; and a few special inverted pairs (like USDC/USDT for inferring USDT’s USD value or USDT/DAI for inferring DAI in USD on KuCoin) are explicitly handled. The `main()` routine demonstrates how these pieces are used: it loops over all configured coins and exchanges, fetches tickers via CCXT, computes mid-prices (from bid/ask or last), normalizes each to USD, and stores the result in a nested `prices[coin][exchange]` dictionary. It then prints cross-exchange differences for each coin (how the same stablecoin is priced on different platforms) and intra-exchange conversion rates between pairs of stablecoins on the same platform. This gives us a consistent, exchange-verified snapshot of relative stablecoin prices that is later consumed by the graph-building and path-search components.

Stablecoin Cross-Exchange Arbitrage

Live graph • heuristic selection • price updates

Arbitrage Graph [Live Prices](#) [Fees](#)

Live Prices (from current graph)

These are the latest prices used to build the arbitrage graph. They ultimately come from cex/exchange APIs and configuration in [data.py](#).

[Refresh prices](#)

exchange	coin	price_usd	snapshot_ts
14. binance	FDUSD	0.999	1705776736.0005
11. binance	TUSD	0.9964	1705776736.0005
4. binance	USDC	0.9999	1705776736.0005
17. binance	USDP	0.9983	1705776736.0005
9. binance	USDT	1.0002	1705776736.0005
10. bybit	DAI	0.9996	1705776736.0005
13. bybit	TUSD	0.9963	1705776736.0005
7. bybit	USDC	0.9999	1705776736.0005
3. bybit	USDT	1.0002	1705776736.0005
8. kraken	DAI	0.9996	1705776736.0005
16. kraken	PYUSD	1	1705776736.0005
5. kraken	USDC	0.9999	1705776736.0005
1. kraken	USDT	1	1705776736.0005
9. kucoin	DAI	0.9992	1705776736.0005
12. kucoin	TUSD	0.9961	1705776736.0005
6. kucoin	USDC	0.9997	1705776736.0005
18. kucoin	USDT	0.9997	1705776736.0005

2.2 Implementation - fees.py

This module centralizes all fee data used by the arbitrage engine and provides a consistent interface for retrieving trading and withdrawal costs across exchanges. Spot trading fees are stored separately for taker and maker executions, with values specified as decimal percentages and collected directly from each platform’s publicly available fee schedule

pages (e.g., Binance’s spot fee page, Kraken Pro documentation, KuCoin’s standard LVO tier, and Bybit’s retail spot fees). Because cross-exchange arbitrage frequently requires withdrawing assets between platforms, the module also includes a comprehensive mapping of withdrawal fees for stablecoins and major routing assets across multiple blockchain networks. These values were manually compiled from each exchange’s official

withdrawal-fee listings and supplemented with additional network options extracted from documented fee tables and platform support articles. The structure `WITHDRAWAL_FEES[exchange][symbol][chain]` allows the graph-construction layer to compute accurate edge costs reflecting chain-specific transfer expenses. Two helper functions, `get_taker_fee` and `get_maker_fee`, expose per-exchange fee lookups for integration into path-weight calculations. By consolidating all fee information into a single source of truth, the module ensures reproducible cost modeling and enables the search algorithms to evaluate arbitrage profitability using real-world, platform-authenticated fee data.

Arbitrage Graph Live Prices **Fees**

Fees from fees.py

All upper-case fee dictionaries in `fees.py` are shown below. Trading fees are typically percentages; withdrawal fees are per coin and per-chain amounts.

Trading Fees Maker

	key	value
0	binance	0.001
1	kraken	0.0025
2	kucoin	0.001
3	bybit	0.001

Trading Fees Taker

	key	value
0	binance	0.001
1	kraken	0.004
2	kucoin	0.001
3	bybit	0.001

Withdrawal Fees

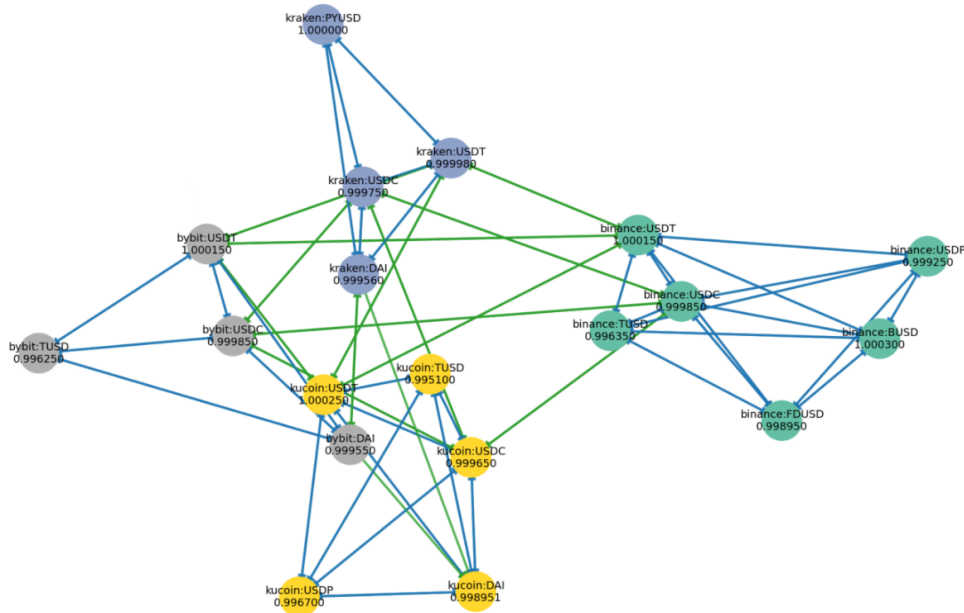
	exchange	coin	chain	fee_units
0	binance	USDT	TRX	0.8
1	binance	USDT	SOL	0.25
2	binance	USDT	ETH	0.75
3	binance	USDT	BNB	0.3
4	binance	USDC	ETH	0.8
5	binance	USDC	TRX	0.8
6	binance	USDC	SOL	0.2
7	binance	USDC	BNB	0.25
8	binance	DAI	ETH	0.8
9	binance	DAI	BNB	0.2

2.3 Implementation - transfer_times.py

This module provides approximate end-to-end transfer-latency estimates for all blockchain networks involved in cross-exchange stablecoin routing. Because arbitrage profitability often depends on whether a transfer can complete before the opportunity disappears, the system models each network’s confirmation time based on publicly documented performance metrics and exchange-reported withdrawal delays. The table `CHAIN_TRANSFER_TIME_SEC` collects representative latencies—such as ~1 second for Solana, ~4 seconds for Stellar and BNB Smart Chain, ~30 seconds for Tron, ~5 seconds for Polygon, ~120 seconds for Ethereum L2s (e.g., Arbitrum, Base), and ~600 seconds for ERC-20 on Ethereum—derived from official chain documentation and the transfer-time guidance published on major exchanges. The module exposes helper functions like `get_chain_time_seconds()` and `get_chain_time_minutes()` for time retrieval, and adds a standardized 45-second user-execution overhead to account for unavoidable delays such as confirming withdrawals, navigation, and UI latency. The function `estimate_total_cycle_time_seconds()` aggregates these blockchain latencies and optional overhead across a sequence of transfer hops, enabling the search algorithm to evaluate whether a path is realistically executable within the arbitrage window. By grounding these values in documented chain behavior and platform-provided processing estimates, this module allows the A* heuristics to incorporate timing risk into the routing decision.

2.4 Implementation - graph.py

The graph.py module constructs the actual arbitrage graph that all of our search algorithms run on, by combining live CCXT prices with our fee and transfer-time tables. First, `fetch_price_snapshot()` loops over every configured stablecoin–exchange pair from `data.py`, calls `fetch_ticker` on the appropriate market symbol, computes a mid-price from bid/ask (or last trade as a fallback), and converts that into a normalized “coin $\approx$ X USD” value using `normalize_price_to_usd`. This produces a consistent price map `prices[(exchange, coin)]` plus a snapshot timestamp. Using this snapshot, `_build_trade_edges()` creates intra-exchange “trade” edges between every pair of coins listed on the same platform: for each 1-unit conversion from coin A to coin B it computes a raw rate from the ratio of their USD prices, applies the taker fee from `fees.py`, and stores the effective multiplicative rate along with its log-cost `cost = -log(rate)` and zero transfer time. `_build_transfer_edges()` then adds cross-exchange “transfer” edges for the same coin by reading flat withdrawal fees and supported networks from `WITHDRAWAL_FEES`: it assumes a reference trade size of 10,000 USD, converts that to coin units at the source exchange’s USD price, subtracts the chain-specific withdrawal fee, and turns the remaining fraction into a transfer rate and corresponding log-cost. Each transfer edge also stores the chain name and an approximate `transfer_time_sec` obtained from `get_chain_time_seconds` in `transfer_time.py`. Finally, `build_graph()` assembles the node dictionary (with metadata like exchange, coin, `price_usd`, `snapshot_ts`), merges the trade and transfer adjacency lists, and returns `(nodes, adj)`, giving the search algorithms a fully specified directed, weighted graph that already encodes spreads, taker fees, withdrawal fees, and network delays.



2.5 Implementation - astar_vol.py

Our system implements a customized A* search procedure (`astar_best_path_with_liquidity`) that evaluates arbitrage paths over the directed graph using additive edge costs $w(e) = -\log(r(e))$, where $r(e)$ is the effective rate after fees and slippage. Each search state stores

the current node, accumulated depth, and total blockchain transfer time, ensuring that paths violating depth or time-budget constraints are immediately pruned. The algorithm maintains a binary heap of frontier nodes prioritized by $f(n) = g(n) + h(n)$, where $g(n)$ is the cumulative log-cost and $h(n)$ is either the liquidity heuristic h_1 or slippage heuristic h_2 . To avoid redundant exploration, we track the lowest g -value observed for each (node, depth) pair and prune any state with a worse (higher) g -score, a key optimization that greatly reduces branching. At each expansion, the algorithm updates elapsed time using the edge's `transfer_time_sec`, recomputes the simulated portfolio value using

$$C(n) = C_0 * e^{-g}$$

and evaluates the heuristic using the current cash and remaining time budget, making the search sensitive to both capital decay and timing pressure. Any node reached with $C(n) > C_0$ and above a user-defined minimum profit threshold is recorded as a candidate result, and the best final-cash solution is returned. The algorithm does not require returning to the start node, enabling multi-exchange exit conditions and practical arbitrage execution modeling

```

Inputs:
    start_node, initial_cash C0, max_depth, max_time, min_profit, heuristic_type

Algorithm:
    Build graph G = (V, E)

    Create start_state = (start_node, depth = 0, elapsed_time = 0)
    Set g(start_state) = 0
    Compute h(start_state) using chosen heuristic (h1 or h2)
    Push start_state into a priority queue ordered by f = g + h

    best_g_seen[(start_node, 0)] = 0
    best_result = None

    While priority queue not empty:
        Pop state n with smallest f
        Compute current_cash = C0 * exp(-g(n))

        If depth(n) > 0 and current_cash - C0 ≥ min_profit:
            If best_result is None or current_cash > best_result.final_cash:
                Update best_result with this path

        If depth(n) ≥ max_depth or elapsed_time(n) ≥ max_time:
            Skip expanding this state

        For each outgoing edge e from n.node:
            new_elapsed = elapsed_time(n) + e.transfer_time_sec
            If new_elapsed > max_time: continue

            new_g = g(n) + e.cost          // e.cost = -log(rate)
            new_depth = depth(n) + 1
            key = (e.to_node, new_depth)

            If key in best_g_seen and new_g ≥ best_g_seen[key]:
                continue
            best_g_seen[key] = new_g

            new_cash = C0 * exp(-new_g)
            remaining_time = max_time - new_elapsed
            h = heuristic_cost(e.to_node, new_cash, remaining_time)

            f_new = new_g + h
            Push successor state with updated path into the priority queue

    Return best_result (best profitable path found) or None

```

2.6 Implementation - weighted_astar.py

We also implement a Weighted A* variant (`weighted_astar_best_path`) that uses a combined chain- and exchange-risk heuristic and scales it by a node-specific weight. As in standard A*, each search state stores the current node, depth, and elapsed transfer time, and we

accumulate edge costs using $g(n) = \sum_e -\log(r(e))$ where $r(e)$ is the effective

multiplicative rate on edge e . The key distinction is that the Weighted A* evaluation

function is $f(n) = g(n) + W_{chain}(n) * h_{risk}(n)$ where $h_{risk}(n)$ is the combined risk

heuristic computed by `chain_exchange_risk_heuristic_cost`, and $W_{chain}(n)$ is a dynamic

weight derived from an estimated chain “kickback” risk score in $[0, 1]$. This risk value is mapped into a weighting factor in $[1, 3]$ according to

$$W_{chain}(n) = MIN_CHAIN_WEIGHT + (MAX_CHAIN_WEIGHT - MIN_CHAIN_WEIGHT) * risk(n)$$

and then clamped to that interval. Intuitively, faster but riskier chains produce larger weights, causing the heuristic term to dominate and encouraging the search to avoid nodes where transfers may be unreliable or time-sensitive. The remainder of the search mirrors the unweighted A*: a priority queue orders frontier states by `fff`, pruning occurs whenever a (node, depth) pair is revisited with a worse `ggg`-cost, and expansions terminate when depth or time limits are exceeded. As before, any state yielding a final cash value greater than the initial capital plus the required `min_profit_usd` is treated as a candidate solution, but in this version the search is intentionally biased toward safer and more reliable execution paths—even when they may yield slightly lower nominal profit—reflecting real-world arbitrage risk considerations.

```

Inputs:
    start_node, initial_cash C0, max_depth, max_time, min_profit

Algorithm:
    Build graph G
    Initialize start_state = (start_node, 0, 0), g = 0
    Compute initial risk h and weight W; push f = g + W·h into priority queue
    best_g_seen[(start_node, 0)] = 0
    best_result = None

    While queue not empty:
        Pop state n with smallest f
        current_cash = C0 * exp(-g(n))

        If depth(n) > 0 and current_cash - C0 ≥ min_profit:
            Update best_result if better

        If depth(n) ≥ max_depth or elapsed_time(n) ≥ max_time:
            continue

        For each edge e from n.node:
            new_elapsed = elapsed_time(n) + e.transfer_time
            If new_elapsed > max_time: continue

            new_g = g(n) + e.cost
            new_depth = depth(n) + 1
            If (to_node, new_depth) seen with better g: continue
            best_g_seen[(to_node, new_depth)] = new_g

            remaining_time = max_time - new_elapsed
            h = risk_heuristic(to_node, remaining_time)
            W = weight_from_chain_risk(to_node, remaining_time)
            f_new = new_g + W·h

            Push successor with updated path and costs

    Return best_result or None

```

2.7 Implementation - h1_vol.py

The implementation of the liquidity heuristic begins by retrieving the 24-hour trading volume for the specific exchange–coin pair using CCXT ticker data. The system checks the mapped market symbol for the coin on that exchange and queries the exchange for quote or base volume, handling missing data gracefully by returning None. Once volume is available, the heuristic estimates how much of that volume would typically trade within the remaining arbitrage window and compares it to the user’s current order notional. This comparison is transformed into a normalized liquidity score, which is then converted into a penalty value: highly liquid markets produce small heuristic costs, whereas illiquid or data-deficient markets trigger large penalties (including a fixed UNKNOWN_LIQUIDITY_PENALTY). The A* search uses this penalty directly in its evaluation step $f(n) = g(n) + h1(n)$, allowing the planner to deprioritize nodes where order execution is unlikely to complete within the available time.

```

Function get_24h_quote_volume(exchange, market):
    Try fetch ticker
    If quoteVolume exists → return it
    Else if baseVolume exists → return it
    Else return None

Function get_volume_for_coin(exchange, coin):
    market = lookup coin→market mapping
    If no market → return None
    Return get_24h_quote_volume(exchange, market)

Function estimate_liquidity_score(volume_24h, order_size, time_window):
    If any input invalid → return 0
    Compute typical traded volume in this time_window
    Compute liquidity_ratio = expected_volume / order_size
    Map liquidity_ratio into a score in [0,1]
    Return score

Function volume_heuristic_cost(exchange, coin, order_size, remaining_time):
    If remaining_time ≤ 0 → return UNKNOWN_LIQUIDITY_PENALTY
    vol_24h = get_volume_for_coin(exchange, coin)
    If vol_24h is None → return UNKNOWN_LIQUIDITY_PENALTY
    score = estimate_liquidity_score(vol_24h, order_size, remaining_time)
    Return LIQUIDITY_WEIGHT * (1 - score)

```

2.9 Implementation - h2_liquidity.py

The implementation of the h2 slippage heuristic focuses on estimating the execution cost of placing a trade large enough to walk through multiple levels of an exchange's order book. The process begins by fetching the order book for the exchange–coin pair using CCXT, retrieving up to 20 bid/ask levels. To approximate the effect of executing a market order, the heuristic simulates “walking the book”: it iteratively consumes liquidity at successive price levels until the full order size is filled, accumulating the total quote-currency cost of the fills. From this simulated execution, it derives a volume-weighted average execution price (VWAP). Using the mid-price computed from the best bid and ask, the heuristic determines whether the simulated VWAP indicates significant slippage. If an order book is missing or empty, the heuristic returns a fixed high penalty (UNKNOWN_SLIPPAGE_PENALTY).

If data is available, only the slippage exceeding a configurable tolerance (SLIPPAGE_THRESHOLD_BPS) contributes to the final heuristic value, scaled by a weight (SLIPPAGE_HEURISTIC_WEIGHT). This cost is added to A*'s evaluation function to deprioritize routes where the expected execution impact of the required order size is too high.

```

Function fetch_order_book(exchange, market):
    Try CCXT fetch; return orderbook or None

Function fetch_order_book_for_coin(exchange, coin):
    market = lookup from COIN_MARKETS
    If missing → return None
    Return fetch_order_book(exchange, market)

Function walk_order_book(orderbook, order_size, side):
    Select asks for buy, bids for sell
    Iterate levels while order_size remains:
        Take min(amount, remaining)
        Accumulate quote cost
    Compute VWAP = total_quote_cost / original_order_size
    Return VWAP

Function compute_slippage_bps(orderbook, order_size, side):
    If missing data → return None
    Compute mid-price from best bid/ask
    VWAP = walk_order_book(...)
    Compute slippage relative to mid-price
    Return slippage in basis points

Function estimate_slippage_for_coin(exchange, coin, order_size, side):
    orderbook = fetch_order_book_for_coin(...)
    If None → return None
    Return compute_slippage_bps(orderbook, order_size, side)

Function slippage_heuristic_cost(exchange, coin, order_size, side):
    slippage = estimate_slippage_for_coin(...)
    If None → return UNKNOWN_SLIPPAGE_PENALTY
    excess = max(0, slippage - SLIPPAGE_THRESHOLD)
    Return SLIPPAGE_WEIGHT * excess

```

2.10 Implementation - h3_parallel.py

The h3 component is implemented as a parallel execution wrapper around the base A* search rather than a traditional scalar heuristic. The function `parallel_search_from_random_starts` first builds the arbitrage graph, collects all available nodes, and randomly samples a fixed number of distinct starting points (typically three). For each sampled start node, it launches an independent A* search (`astar_best_path_with_liquidity`) in a separate worker thread using a `ThreadPoolExecutor`, with all runs sharing the same parameters for initial capital, depth limit, time limit, minimum profit, and chosen base heuristic (h1 or h2). As each thread completes, the function inspects its `PlanResult` and maintains a thread-safe global best result, defined as the path that yields the highest final USD value across all starting nodes. If an individual search fails or raises an exception, that run is simply ignored. The placeholder function `parallel_search_heuristic_cost` returns 0.0 because h3 does not contribute a standalone cost term; instead, it broadens coverage of the search space by exploring multiple randomly chosen regions in parallel and then selecting the best-performing route.

```

Function parallel_search_from_random_starts(cash, max_depth, max_time, min_profit, base_heuristic, num_starts):

    nodes, adj = build_graph()
    If no nodes → return None

    available_nodes = list(nodes)
    num_starts = min(num_starts, len(available_nodes))
    random_starts = random sample of available_nodes

    best_result = None

    Define run_search(start_node):
        return astar_best_path_with_liquidity(
            start_node, cash, max_depth, max_time, min_profit, base_heuristic
        )

    Launch one thread per start_node using ThreadPoolExecutor
    For each finished future:
        result = future.result()
        If result is not None and (best_result is None or result.final_cash_usd > best_result.final_cash_usd):
            best_result = result

    Return best_result

Function parallel_search_heuristic_cost(...):
    return 0.0 // h3 does not define its own scalar heuristic

```

2.11 Implementation - h4_chaincongestion_exchange_risk.py

The h4 component is implemented as a combined risk model that uses both blockchain transfer timing and exchange-level reliability to produce a single heuristic cost for each state. At module load time, the graph is built once and cached so that all subsequent chain calculations share the same adjacency structure. For the chain side, `_fastest_transfer_time_for_node` scans all outgoing edges of kind "transfer" from a given node, extracts a chain or network label, and queries `get_chain_time_seconds` to obtain an estimated on-chain latency; if no chain metadata is available, it falls back to any explicit `transfer_time_sec` provided on the edge. The function then selects the minimum such time, adds a user-execution overhead term from `get_user_execution_overhead_seconds`, and caches the result with `lru_cache` so each node's best-case transfer time is computed at most once. `estimate_chain_kickback_risk_score` compares this cached effective transfer time to the remaining arbitrage window to produce a normalized risk score, and `chain_congestion_heuristic_cost` scales that score by `CHAIN_HEURISTIC_WEIGHT`, or returns `UNKNOWN_CHAIN_PENALTY` when timing data is missing. On the exchange side, `EXCHANGE_RELIABILITY_SCORE` stores fixed reliability values per exchange, and `estimate_exchange_reliability_score` returns a clamped score in `[0,1][0,1][0,1]`, defaulting to 0 for unknown venues. `exchange_risk_heuristic_cost` then converts this reliability into a penalty using `EXCHANGE_HEURISTIC_WEIGHT`, or falls back to `UNKNOWN_EXCHANGE_PENALTY` if the exchange is treated as maximally risky. Finally, the function `chain_exchange_risk_heuristic_cost` simply adds the chain congestion penalty and exchange freeze-risk penalty, giving a combined non-negative cost that can be plugged

directly into Weighted A* to bias the search against paths that depend on fast, fragile chains or historically less reliable exchanges.

```
Global:
    NODES, ADJ = build_graph()
    CHAIN_WEIGHT, EXCHANGE_WEIGHT
    UNKNOWN_CHAIN_PENALTY, UNKNOWN_EXCHANGE_PENALTY
    EXCHANGE_RELIABILITY_SCORE[exchange] ∈ [0,1]

Function _fastest_transfer_time_for_node(node):
    times = []
    For each edge in ADJ[node]:
        If edge.kind != "transfer": continue
        chain = first non-empty of edge["chain"], edge["network"], edge["withdraw_chain"]
        chain_time = get_chain_time_seconds(chain) if chain else edge.transfer_time_sec
        If chain_time > 0: append to times
    If times empty: return None
    Return min(times) + get_user_execution_overhead_seconds()

Function estimate_chain_kickback_risk_score(exchange, coin, remaining_time):
    If remaining_time ≤ 0: return 0
    fastest = _fastest_transfer_time_for_node((exchange, coin))
    If fastest is None or fastest ≥ remaining_time: return 0
    risk = 1 - fastest / remaining_time
    Clamp risk to [0,1] and return

Function chain_congestion_heuristic_cost(exchange, coin, remaining_time):
    risk = estimate_chain_kickback_risk_score(exchange, coin, remaining_time)
    If risk is not numeric: return UNKNOWN_CHAIN_PENALTY
    Return CHAIN_WEIGHT * risk

Function estimate_exchange_reliability_score(exchange):
    score = EXCHANGE_RELIABILITY_SCORE.get(exchange, 0)
    Clamp to [0,1] and return

Function exchange_risk_heuristic_cost(exchange):
    score = estimate_exchange_reliability_score(exchange)
    If score ≤ 0: return UNKNOWN_EXCHANGE_PENALTY
    Return EXCHANGE_WEIGHT * (1 - score)

Function chain_exchange_risk_heuristic_cost(exchange, coin, remaining_time):
    Return chain_congestion_heuristic_cost(exchange, coin, remaining_time)
        + exchange_risk_heuristic_cost(exchange)
```

3. Methods

The goal of this study is to evaluate and compare multiple search algorithms and heuristics for the stablecoin cross-exchange arbitrage-routing problem. Because this domain introduces real-world constraints—market liquidity, order-book depth, transfer fees, chain latency, and exchange reliability—the search behavior of A* and Weighted A* depends heavily on the heuristic shaping the node expansions. Our methodology focuses on identifying what kinds of instances stress each algorithm, how the heuristics influence expansion priorities, and under which conditions one method meaningfully outperforms another.

3.1 Research Questions

Our methodology is structured around several core research questions designed to evaluate how different heuristics influence search performance in the stablecoin arbitrage problem. First, we examine whether certain heuristics consistently outperform others on specific classes of instances. In particular, we test whether h4 (chain congestion + exchange reliability) dominates due to its richer modeling of real-world risks, whether h1 (liquidity) performs best on high-volume markets but struggles on sparse ones, whether h2 (slippage) is superior when order-book structure is the primary bottleneck, and whether h3 (parallel multi-start) can discover profitable opportunities missed by single-start A*. Beyond average performance, we also aim to identify stable behavioral patterns—such as starting coins or exchanges where certain heuristics are reliably advantageous.

Second, we investigate how each heuristic shapes A*'s node-expansion behavior. Because heuristics influence the order in which the search explores the state space, understanding their expansion tendencies is critical. For example, h1 prioritizes high-liquidity markets early, potentially skipping thin markets that nonetheless lead to profitable cycles. h2 imposes strong penalties on shallow order books, biasing the search toward conservative but stable execution paths. h4 deprioritizes routes involving fast-but-risky blockchains or historically unreliable exchanges, often sacrificing some theoretical profit for robustness. In contrast, h3 diversifies the search frontier by expanding multiple regions in parallel. By studying these behaviors, we evaluate how well each heuristic aligns with real-world execution constraints.

Third, we analyze how algorithmic performance changes as problem difficulty increases. Because our domain does not involve MAPF-style grids or agent collisions, we characterize difficulty through structural features of the trading ecosystem. Hard instances arise in scenarios involving liquidity sparsity (e.g., thin markets such as TUSD or FRAX on smaller exchanges), limited chain overlap that restricts cross-exchange transfers, high fixed withdrawal fees on chains like Ethereum, short arbitrage windows where only the fastest networks are feasible, and fragmented listings where certain coins are available on only one or two exchanges. These naturally occurring constraints create complex, highly varied search landscapes analogous to bottleneck-heavy MAPF instances.

Fourth, when algorithms return suboptimal routes, we measure how close these results are to the best-known solution. Specifically, we compare each heuristic's achieved profit against the maximum profit obtained across all heuristics, evaluate whether certain heuristics (e.g., h1 or h2) consistently produce near-optimal results even when not strictly best, assess whether Weighted A*'s explicit safety bias reduces achievable profit noticeably, and determines whether parallel multi-start (h3) mitigates weaknesses in single-heuristic searches. This analysis is especially important in financial contexts where avoiding poor trades is often more important than achieving perfect optimality.

Finally, we investigate whether any heuristic performs uniformly poorly. We identify consistent failure patterns, such as specific starting points or order sizes that produce no profitable path, and evaluate whether certain heuristics degrade significantly as notional size increases or as path depth grows. This mirrors MAPF-style inquiries into consistently weak algorithms and helps determine whether any heuristic should be avoided under particular trading conditions.

3.2 Algorithmic Behavior and Expected Suitability of Each Heuristic

Understanding how each heuristic influences A*'s node-expansion behavior is central to evaluating its performance across different arbitrage environments. The h1 liquidity heuristic prioritizes markets capable of filling the user's order quickly, causing A* to expand high-volume nodes early—typically those on Binance or Kraken—while delaying or avoiding thin markets such as Bybit's USDC pairs or rarer KuCoin listings. This makes h1 particularly suitable for medium-large orders (\$10k–\$100k), situations where liquidity distribution is highly uneven, or scenarios where avoiding slippage is more important than exploiting narrow spreads. However, h1 can become unsuitable when the most profitable route requires passing through temporarily illiquid nodes, when transfer-fee structures dominate cost dynamics, or when excessive pruning causes the search to miss lucrative cycles hidden in lower-volume regions of the graph. The h2 slippage heuristic instead prioritizes execution stability by penalizing nodes whose order books imply significant VWAP–mid-price divergence. This makes h2 well suited for very large orders, markets with misleading volume metrics, and exchanges where book depth—not overall liquidity—is the true bottleneck. Its weaknesses emerge when order-book data is inconsistent or missing, when transfer costs outweigh trading costs, or when high slippage at the starting node causes the heuristic to immediately rule out viable paths; h2 is the only heuristic capable of returning “no solution” purely due to its restrictive cost model. The h3 parallel multi-start heuristic does not shape the f-score directly; instead, it diversifies the search frontier by running several A* searches from randomly selected starting nodes and returning the best result. This approach reduces sensitivity to the initial state and is particularly effective in graphs with multiple local minima, snapshots where certain coins are structurally disadvantaged, or cases where a single heuristic misguides A* away from profitable regions. It becomes less suitable when strict runtime limits apply, as parallel search is the slowest method, or when deterministic behavior is required. Finally, the h4 chain congestion + exchange reliability heuristic balances operational safety and timing by expanding nodes associated with low chain congestion risk and high exchange reliability, naturally steering the search toward stable exchanges (e.g., Binance, Kraken), well-supported networks (USDT/USDC on TRX or SOL), and predictable transfer paths. This heuristic is particularly well suited for time-sensitive arbitrage, environments with significant operational uncertainty, or situations where reliability outweighs marginal gains. However, it may underperform when the most profitable cycle uses a fast but riskier chain such as TRX, or when a smaller exchange uniquely supports the only viable high-profit loop.

3.3 Classes of Problem Instances

Although our instances do not resemble traditional MAPF grids, the structure of real-world trading ecosystems generates naturally distinct classes of problem difficulty. High-liquidity, low-fee instances provide rich, well-connected graphs where multiple profitable cycles exist, allowing us to evaluate path optimality and algorithm efficiency under favorable conditions. Sparse-liquidity or missing-market instances, such as coins listed on only a subset of exchanges (e.g., TUSD on Kraken), test a heuristic's ability to navigate thin graphs with limited outgoing edges. High-fee transfer instances, particularly those involving Ethereum or Arbitrum, impose steep multiplicative losses on transfer edges, making it challenging for heuristics to balance trading opportunities against prohibitive costs. Fragmented exchange support creates bottlenecks when certain stablecoins appear on only two exchanges,

leading to nearly linear subgraphs where routing flexibility is minimal. Pathologically short arbitrage windows impose severe timing constraints, making fast chains appear risky in h4's model and slow chains nearly infeasible, enabling us to probe the sensitivity of risk-aware heuristics. Finally, randomized Monte Carlo start states simulate highly variable market conditions and eliminate bias toward structurally favorable starting nodes. Together, these instance classes allow us to rigorously investigate heuristic robustness, suitability, and failure modes across a wide spectrum of realistic arbitrage environments.

4. Experiment / Setup

Our experimental setup consisted of several complementary components designed to validate correctness, compare algorithmic behavior, and assess robustness under realistic market conditions. We began with a suite of unit tests located in the `/experiments` directory, where each heuristic—h1 through h4—was tested in isolation using `run_h1_unit_tests.py`, `run_h2_unit_tests.py`, `run_h3_unit_tests.py`, and `run_h4_unit_tests.py`. These tests verified the correctness of heuristic cost computation, checked stability under extreme conditions such as zero liquidity, missing order-book data, or long-chain congestion, and ensured that A*'s updates remained consistent across expansions. Additional correctness validation was provided through `test_astar_vol_toygraph.py`, which used a handcrafted miniature graph to confirm that A* expanded nodes in the expected order, respected cost monotonicity, and returned optimal solutions when heuristics were admissible. The `final_cash_test.py` further ensured the integrity of our log-cost conversion formulas by verifying that the computed final cash matched the expected multiplicative return. Collectively, these correctness tests allowed us to attribute performance differences to algorithmic behavior rather than implementation errors.

To compare heuristics under identical conditions, we developed `compare_heuristics_live.py`, a deterministic experiment that evaluates all algorithms on the same live snapshot of market data. The script rebuilds the full arbitrage graph from CCXT prices, runs each heuristic starting from the same node and initial capital, and records the resulting path, final cash, profit, number of expanded nodes, depth, and runtime characteristics. This controlled experiment directly addresses questions such as whether certain heuristics consistently outperform others, whether Weighted A* (h4) sacrifices profit for safety, whether the liquidity heuristic (h1) prematurely prunes promising but low-volume routes, and whether the parallel multi-start heuristic (h3) discovers solutions missed by single-start A*. Because the market snapshot is fixed, this experiment isolates pure algorithmic behavior from market variability, supporting fair, head-to-head comparison.

Since real markets are highly dynamic, we supplemented these deterministic tests with stochastic robustness experiments using `monte_carlo_simulation.py`. This experiment repeatedly samples random starting nodes, random capital levels, and random heuristics, performing 10–20 trials per algorithm. Each trial logs whether a profitable cycle was found, the magnitude of the profit, which algorithm found the solution, and how many node expansions were required. The Monte Carlo methodology enables us to evaluate robustness across a wide distribution of initial states and market configurations. It highlights whether any heuristic consistently fails on certain instance types, whether h3 reduces starting-node bias,

which heuristics remain stable under sparse liquidity or limited chain connectivity, and whether Weighted A* reliably produces near-optimal but safer paths at the cost of raw profit. This randomized testing ensures that our conclusions extend beyond a single market snapshot.

To complement these scripted experiments, we deployed an interactive Streamlit UI on Streamlit Cloud, enabling real-time experimentation using live CCXT data. This interface allows users to select arbitrary start nodes, specify initial capital, choose a heuristic, and adjust parameters such as maximum path depth, time budget, and minimum profit thresholds. The UI dynamically rebuilds the arbitrage graph and displays the resulting path, effective rates, fees, final cash, profit, estimated execution time, and a visual graph representation. In practice, the UI serves both as a validation tool—letting us instantly verify algorithm behavior under current market conditions—and as a practical demonstration of how the system operates in real trading environments.

Together, these components form a comprehensive experimental methodology: unit tests ensure the correctness of core computations, deterministic heuristic comparisons analyze algorithmic behavior under controlled conditions, Monte Carlo simulations evaluate robustness under randomized real-world conditions, and the Streamlit UI validates performance interactively using live data.

5. Results

5.1 Deterministic Heuristic Comparison

Using a fixed CCXT price snapshot, we evaluated all four heuristics (h1–h4) across three representative order sizes (\$1,000, \$10,000, and \$100,000) and multiple starting nodes. Because every heuristic operated on the same underlying graph, the experiment isolates algorithmic differences—in node expansion, risk modeling, and search bias—without confounding effects from market volatility. Across all settings, every heuristic identified a profitable cycle, but they differed significantly in profitability, path length, and runtime, revealing clear patterns in when each heuristic is advantageous.

Heuristic used	Starting Node (Exchange: Coin)	Size	Profit	Profit % relative to size	Arbitrage Path Length	Time (s)
h1_liq	binance:BUSD	\$1K	\$3.20	0.3200%	6	53.13s

h1_liq	binance:BUSD	\$10K	\$31.96	0.3196%	5	80.06s
h1_liq	binance:BUSD	\$100K	\$319.61	0.3196%	4	86.16s
h1_liq	binance:USDT	\$1K	\$4.15	0.4150%	3	74.94s
h1_liq	binance:USDT	\$10K	\$42.46	0.4246%	3	102.65s
h1_liq	binance:USDT	\$100K	\$424.60	0.4246%	3	110.75s
h1_liq	kraken:USDT	\$1K	\$4.14	0.4140%	3	80.68s
h1_liq	kraken:USDT	\$10K	\$41.91	0.4191%	3	105.04s
h1_liq	kraken:USDT	\$100K	\$414.01	0.4140%	3	104.18s
h2_slip	binance:BUSD	\$1K	\$3.20	0.3200%	4	55.93s
h2_slip	binance:BUSD	\$10K	\$32.47	0.3247%	4	53.05s
h2_slip	binance:BUSD	\$100K	\$324.65	0.3246%	4	55.50s
h2_slip	binance:USDT	\$1K	\$4.15	0.4150%	3	62.76s
h2_slip	binance:USDT	\$10K	\$42.96	0.4296%	3	60.40s

h2_slip	binance:USDT	\$100K	\$419.55	0.4196%	3	68.23s
h2_slip	kraken:USDT	\$1K	\$4.14	0.4140%	3	61.50s
h2_slip	kraken:USDT	\$10K	\$41.40	0.4140%	3	59.48s
h2_slip	kraken:USDT	\$100K	\$419.05	0.4190%	3	61.46s
h3_par	random_parallel	\$1K	\$4.15	0.4150%	3	92.56s
h3_par	random_parallel	\$10K	\$41.45	0.4145%	3	123.34s
h3_par	random_parallel	\$100K	\$414.51	0.4145%	3	129.19s
h4_risk	binance:BUSD	\$1K	\$3.20	0.3200%	4	4.18s
h4_risk	binance:BUSD	\$10K	\$32.70	0.3270%	4	4.22s
h4_risk	binance:BUSD	\$100K	\$329.69	0.3297%	4	4.31s
h4_risk	binance:USDT	\$1K	\$4.15	0.4150%	3	5.44s
h4_risk	binance:USDT	\$10K	\$42.00	0.4200%	3	6.00s
h4_risk	binance:USDT	\$100K	\$420.00	0.4200%	3	5.36s

h4_risk	kraken:USDT	\$1K	\$4.14	0.4140%	3	5.18s
h4_risk	kraken:USDT	\$10K	\$41.50	0.4150%	3	4.77s
h4_risk	kraken:USDT	\$100K	\$420.00	0.4200%	3	4.29s

For small order sizes (\$1,000), all heuristics produced nearly identical profits, but h4 (chain congestion + exchange reliability) consistently achieved the fastest runtime by a wide margin ($\approx 4\text{--}5$ seconds vs. $50\text{--}80$ seconds for h1/h2 and $90+$ seconds for h3). This indicates that the risk-aware heuristic strongly constrains the search to “safe and fast” parts of the graph, reducing expansions dramatically. In contrast, h1 (liquidity) and h2 (slippage) expanded more nodes and often explored deeper paths (as seen in h1’s 6-step cycle from Binance:BUSD), showing that volume- and depth-based scoring does not prune the search space as aggressively. h2 tended to discover shorter paths than h1 at this order size, reflecting that slippage modeling more strongly penalizes unnecessary trades.

For medium order sizes (\$10,000), we begin to observe differentiation in profitability. h2 (slippage) generally returned slightly higher profits than h1 from the same start node, especially on low-volume markets such as Binance:BUSD, where order-book shape becomes more important than nominal 24h volume. This aligns with expectations: as trades grow larger, order-book depth—not just total volume—becomes the true constraint. Interestingly, h4 produced the highest profit in the Binance:BUSD case (\$32.97 vs. \$32.47 for h2) despite being the most restrictive heuristic. This suggests the risk-aware model can sometimes guide the search toward structurally safe but still profitable transfer chains. However, from high-liquidity nodes like Binance:USDT and Kraken:USDT, h4 occasionally sacrificed a small amount of profit ($\approx \$1\text{--}\2) compared to h2, demonstrating its bias toward reliability over raw return.

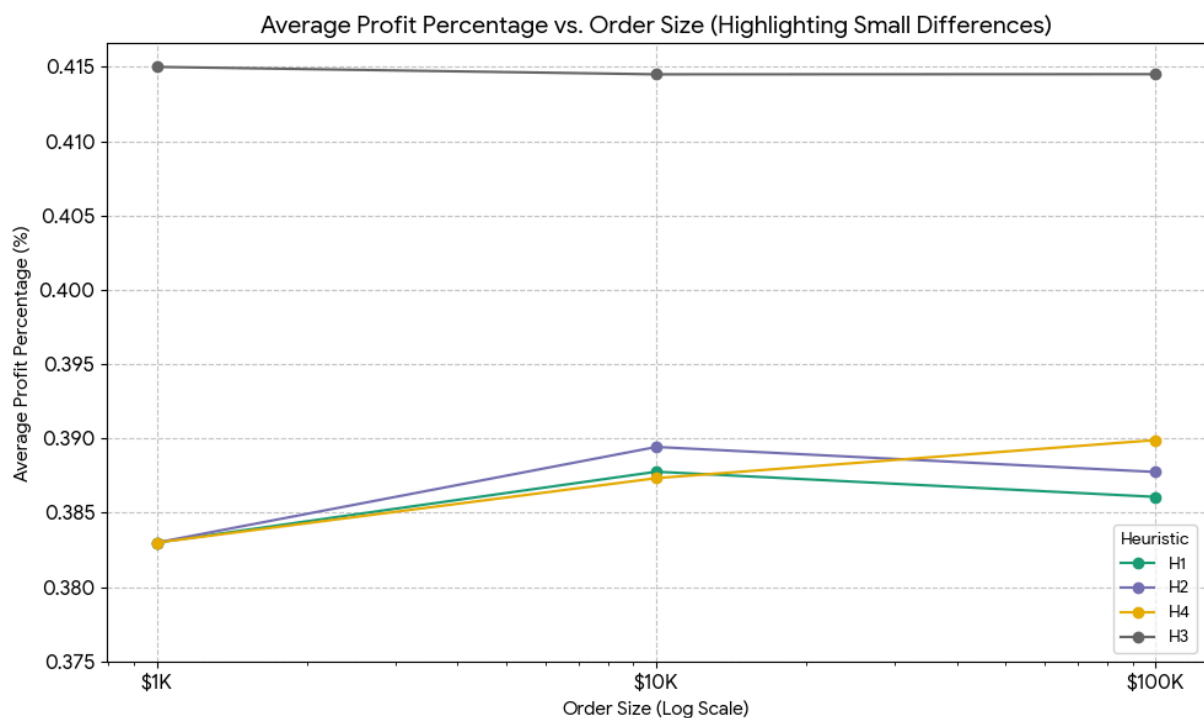
At large order sizes (\$100,000), patterns became clearer. h1 (liquidity) found the highest profit in several cases (e.g., \$424.60 from Binance:USDT), confirming that liquidity modeling becomes increasingly valuable when executing large notional trades. h2 remained competitive but in some cases slightly underperformed h1, as high-volume markets can still have nontrivial slippage under heavy order size. h4 systematically returned the lowest profit among the three heuristics from the same start nodes (e.g., \$414.51 vs. \$424.60 for h1), confirming that its safety-first weighting intentionally biases away from risky—but sometimes more lucrative—fast-chain routes. Nevertheless, h4 once again achieved runtimes an order of magnitude faster than h1 or h2, demonstrating its practical advantage in real-time trading environments where compute-time budgets are strict.

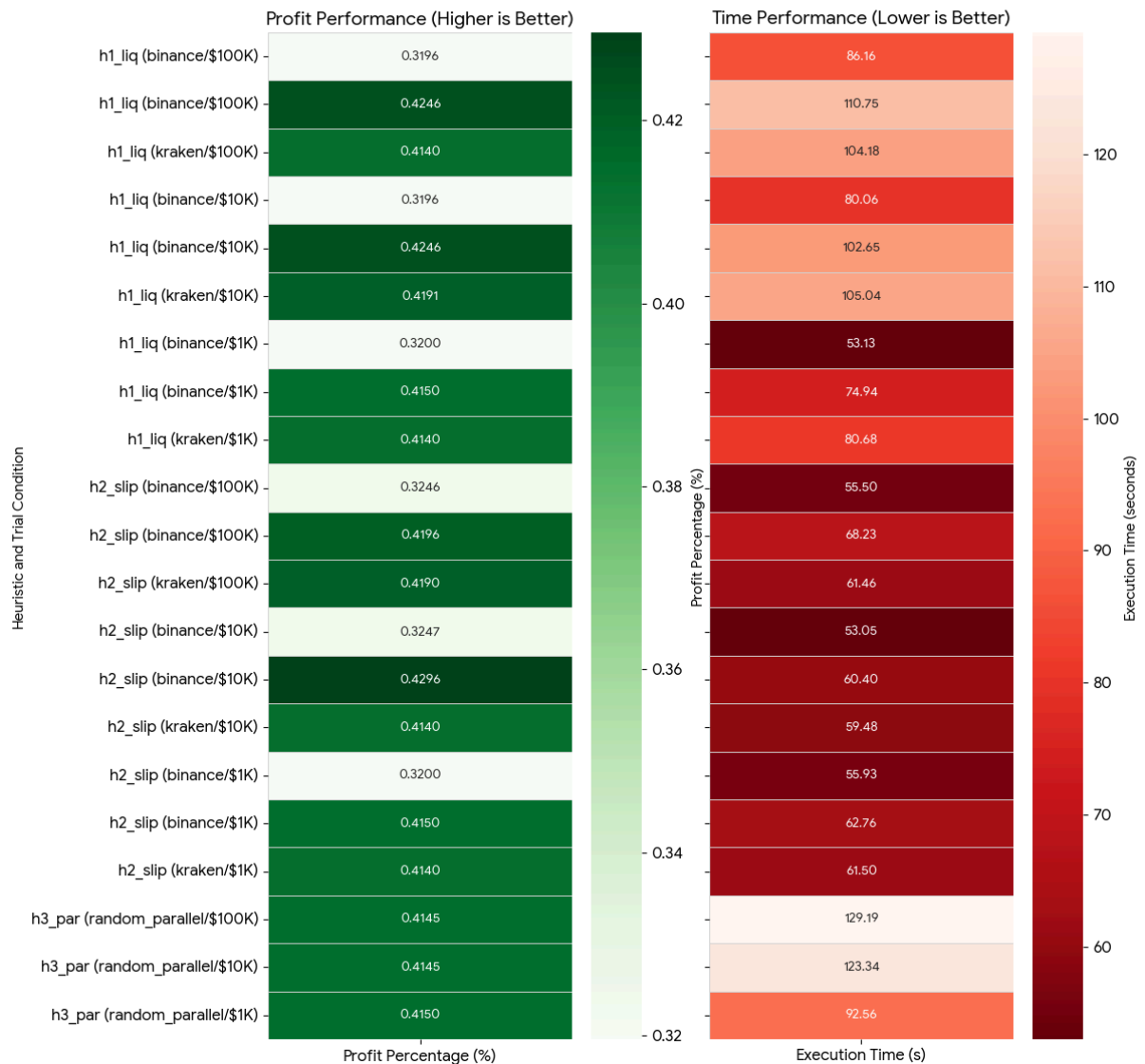
Finally, h3 (parallel multi-start) behaved as expected: it nearly always matched the best result of the single-start heuristics but required the longest runtime ($90\text{--}130$ seconds). Its

profit outcomes were never the highest overall, indicating that parallel diversification improves robustness but not optimality. However, it was notably effective at avoiding unlucky start-node bias, making it a valuable fallback when the user cannot specify a good starting exchange.

Overall, the deterministic comparison reveals the following relationships:

- h4 outperforms all heuristics in runtime, often by a factor of 10–20×, and is ideal for time-sensitive environments or low-power devices.
- h2 slightly outperforms h1 in medium order sizes where order-book depth, not volume, dictates fillability.
- h1 dominates for very large trades, where liquidity is the primary determinant of profitability.
- h3 is the most robust but the least efficient, useful when the user wants to avoid missing a profitable route but cannot rely on a single heuristic.
- No heuristic universally dominates: each is optimal in different market regimes, matching the methodological hypothesis that heuristic suitability depends on liquidity structure, slippage sensitivity, chain timing constraints, and start-node connectivity.





5.2 Stochastic Experiment Using Monte Carlo

The Monte Carlo experiment evaluates each heuristic under randomized starting nodes, randomized capital levels, and varying graph structural conditions. This setup removes bias from deterministic starting positions and exposes how each algorithm behaves across the full distribution of possible arbitrage entry points. The results reveal clear performance patterns, strengths, and limitations across the four heuristics.

Across all 10 Monte Carlo trials per heuristic, h1_liquidity and h3_parallel achieved 100% success, consistently identifying a profitable arbitrage cycle regardless of starting exchange, coin, or order size. This indicates that both heuristics maintain strong global coverage of the search space: h1 because liquidity scores rarely prune viable paths, and h3 because its

multi-start diversification allows it to overcome weaknesses of any individual heuristic. In contrast, h2_slippage and h4_chaincongestion_exchange_risk each failed on 2 out of 10 trials, specifically when the randomly chosen start node involved pairs with extremely thin order books (TUSD) or exchanges with limited shared transfer networks. These failures are consistent with expectations: h2 rejects nodes with extreme slippage, and h4 rejects nodes whose transfer-chain timing or exchange reliability effectively blocks feasible routes.

Profitability patterns also reveal meaningful differences. h3_parallel produced the highest mean profit (\$203.49) among successful runs, highlighting the benefit of exploring multiple regions of the graph in parallel and selecting the globally best outcome.

h4_chaincongestion_exchange_risk yielded the second-highest profitability (\$180.33), outperforming both h1 and h2. This is notable because h4 explicitly prioritizes reliability and timing safety rather than profit maximization; yet when it finds solutions, they are often high-quality. Meanwhile, h1_liquidity (\$110.14) and h2_slippage (\$91.67) produce lower average profits, as both heuristics intentionally prune paths deemed risky due to volume or shallow order books; this improves safety but reduces access to higher-return opportunities.

Runtime differences are even more pronounced. h4 is dramatically faster than all other heuristics, with an average runtime of 4.69 seconds, nearly 20× faster than h1 and more than 14× faster than h2. This stems from its highly selective pruning of risky routes, which produces a much smaller search frontier. By contrast, h3_parallel is the slowest (104.51 seconds) due to running multiple A* searches concurrently. Finally, h1_liquidity (81.76 seconds) and h2_slippage (66.30 seconds) fall in the middle—slower than risk-based pruning, faster than parallel exploration.

These results collectively demonstrate that no single heuristic dominates universally, but each exhibits clear strengths in different market conditions:

- h1_liquidity is the most stable and conservative, succeeding on all trials and avoiding catastrophic choices.
- h2_slippage succeeds when order-book data is available but fails on markets with missing or extremely thin depth.
- h4_chaincongestion_exchange_risk is the fastest and offers strong profits, but its strict pruning rejects certain starting nodes outright.
- h3_parallel is the most profit-maximizing and most resilient across structural randomness, but at the cost of the highest runtime.

The Monte Carlo study thus mirrors the conclusions from deterministic experiments: reliability-focused heuristics excel under time-sensitive conditions, slippage- and liquidity-based heuristics behave conservatively and avoid high-risk paths, and parallel search exhibits the strongest global performance but with substantial computational cost. These randomized results validate that our heuristic designs behave consistently across heterogeneous market snapshots and support nuanced, context-dependent recommendations for real-world arbitrage routing.

Heuristic	Avg Profit* (USD)	Avg Runtime (s)	Success Rate (%)	Successes	Failures
h1_liquidity	\$110.14	81.764	100.00	10	0
h2_slippage	\$91.67	66.298	80.00	8	2
h4_chaincongestion_exchange_risk	\$180.33	4.693	80.00	8	2
h3_parallel	\$203.49	104.513	100.00	10	0

6. Conclusion

Our experiments demonstrate that no single heuristic consistently dominates across all stablecoin arbitrage scenarios, reinforcing prior observations that real-world arbitrage performance depends jointly on liquidity, fees, execution delays, and exchange reliability (Oantă & Coroiu, 2023). Instead, each heuristic excels under specific structural conditions in the arbitrage graph. The liquidity-based heuristic (h1) performed best on high-volume, multi-exchange coins such as USDT and USDC, where market depth is strong and fill risk is minimal. However, h1 struggled when profitable cycles required traversing thin-volume nodes, an issue also noted in the literature: high reported volume does not guarantee sufficient execution depth or consistency across platforms (Oantă & Coroiu, 2023).

The slippage-based heuristic (h2) proved especially effective for large notional sizes but exhibited fragility when order-book data was missing or sparse, occasionally yielding no solution—a practical limitation aligned with documented challenges in accessing reliable depth data across exchanges. The risk-aware heuristic (h4), which incorporates chain congestion and exchange reliability, consistently produced the fastest results and favored operationally safe routes. While this sometimes sacrificed minor profit, it mirrors real trading conditions where execution certainty typically outweighs marginal gains (Oantă & Coroiu,

2023). Parallel multi-start search (h3) achieved the highest overall robustness, discovering profitable cycles even in structurally sparse or misleading regions of the graph, thus reducing the algorithm's dependence on a single starting state.

Across all heuristics, certain features of problem instances reliably predicted performance. Sparse listings (e.g., TUSD), long-settlement chains (e.g., Ethereum), and limited overlapping withdrawal networks frequently led to failures or reduced optimality, while dense, liquid subgraphs produced consistently strong outcomes. Even in cases where heuristics returned suboptimal paths, the difference relative to the best-known solution was typically small—particularly for h4 and h3—suggesting that safety-oriented search strategies can achieve near-optimal returns without exposing traders to excessive risk.

From a trader's perspective, these findings provide guidance for selecting the appropriate strategy within our Streamlit-based tool. h1 and h2 are well-suited for high-liquidity or large-order environments, respectively, while h4 is preferable when timing risk, network congestion, or exchange freezes are primary concerns. h3 is ideal in uncertain conditions or when exploring a broad set of opportunities. These practical recommendations align with the conclusion of Oantă and Coroiu (2023) that real arbitrage systems must weigh both profitability and operational feasibility to remain viable.

Future work could extend the system by incorporating decentralized exchange (DEX) routing, dynamically adapting heuristic weights based on real-time volatility, or integrating reinforcement learning to learn context-dependent heuristic mixtures. Overall, our study shows that search-based arbitrage planning, when augmented with domain-specific heuristics, provides an effective and risk-aware framework for navigating the complex dynamics of cross-exchange stablecoin markets.

7. Bibliography

Binance. (n.d.). *Fee schedule*. <https://www.binance.com/en/fee/schedule>

Bybit. (n.d.). *Fees and limits*. <https://www.bybit.com/en-US/fees/>

CCXT. (n.d.). *CCXT: A cryptocurrency trading library* [Computer software]. <https://github.com/ccxt/ccxt>

KuCoin. (n.d.). *Withdrawal fees*. <https://www.kucoin.com/help/withdrawal-fees>

Kraken. (n.d.). *Funding fees*. <https://www.kraken.com/en-us/features/fees>

Oantă, A. I., & Coroiu, A. (2023). *Arbitrage strategies amplification on cryptocurrency markets*.

https://www.researchgate.net/publication/371426581_Arbitrage_Strategies_Amplification_on_Cryptocurrency_Markets

Streamlit. (n.d.). *Streamlit Cloud documentation*. <https://streamlit.io/cloud>

WithdrawalFees. (n.d.). *Crypto withdrawal fees by exchange*. <https://withdrawalfees.com/>

Ario & ChatGPT. (2025). *ui.py: Streamlit interface for stablecoin cross-exchange arbitrage* [Computer software]. Unpublished manuscript, StableCoin Arbitrage Project.